Data API

Compatible With Taopix Control Centre v2021r2+

Author: Taopix Support
Date: 30/03/2021
Document Name: Taopix Data API v12.0.1

PRIVATE & CONFIDENTIAL

Revision History

2nd February	V9.0.0	Taopix Data API v9.0.0 document created. Separated out from Taopix Web Shopping Cart Breakout documentation.
27th June 2018	V10.0.0	Added the checkDataAvailable function to allow the availability of the order files to be queried.
1st August 2019	V11.0.0	Changed random number generation to use and support CSPRNG binary outputs for usage in Initialisation Vectors as well as support for new IVs to be generated for the return transmission by Control Centre. Debug mode now defaults to zero in the clientSideLibrary.php Delete queries now return an error if the system has not found any rows to delete Added the library function getProjectOrderData to retrieve project order data required to insert an order into the database. Added the library function insertOrder to insert an entire multi line order into the database.
25th April 2020	V12.0.0	Standardised the result format for Query, Insert, Delete and Update queries. Added the 'projectaimode' column on ORDERITEMS to allow aimode tracking. Added the function 'insertApiCall' to format the result of a single insert action.
30th March 2021	V12.0.1	Added the insertOrderRetroPrintsComponent function to allow inserting Retro print components.

INTRODUCTION
COMMUNICATING WITH TAOPIX CONTROL CENTRE

The Taopix Control Centre Data API is a secure mechanism to allow the interaction between external systems and the Taopix system without having a direct connection to the Taopix database. The Data API currently allows the following functions to be performed:

1. Database query commands

2. Database manipulation commands (insert / update / delete)

3. API calls to authenticate, update status information, close / cancel sessions etc…

The Data API uses a HTTP and JSON to receive commands and to return results. To prevent spoofing and man-in-the-middle attacks the API requires authentication and uses Blowfish encryption and unique transaction identifiers to provide a secure channel between the external system and Taopix Control Centre.

The Data API Life Cycle

1. Authentication
   Before any data query / data manipulation or other API calls can be executed the external system must authenticate with Taopix Control Centre. Successful authentication will generate a web session that is used for future calls.

2. Execute Commands
   After successfully authenticating the external system can execute the various API calls to query and manipulate the Taopix database.

3. End Session
   This API call is executed when the session is not required any more.

Data API Communication

The DATA API endpoint is: `http://taopix.server/?fsaction=AppDataAPI.`*`apicommand`*

The API commands are as follows:

Authenticate	Create and authenticate a new Data API session
TestSession	Test a session to confirm that it is still valid
EndSession	End a session when it is no longer required
DeleteSession	Delete a session immediately
Query	Query a Taopix database table
Insert	Insert a record into a Taopix database table
Update	Update an existing record in a Taopix database table
Delete	Delete a record from a Taopix database table
Execute	Execute a SQL command against the Taopix database
TriggerOrder	Trigger a new order after it has been inserted into the Taopix database
CancelOrder	Informs Taopix that the external shopping cart session has been cancelled.
CheckDataAvailable	Checks that the order data files are available for an order item. This should be invoked before a reorder.
getProjectOrderData	Retrieves project order data required to insert an order into the database.
insertOrder	Insert an entire multi line order into the database.

Although the Data API uses HTTP and JSON to transmit and receive the API call the contents of the API call will be encrypted. Depending on the configuration of the system the result of the API call may also be encrypted. The encryption method is Blowfish using CBC mode and a random initialisation vector. The encryption secret must be configured both on the Taopix Control Centre server and on the server that is using the Data API.

NOTES

1. Integrators that are using PHP for their external shopping cart can use the provided `clientSideLibrary.php` class to perform the low-level communication. Integrators that are not using PHP will have to re-implement this functionality in their chosen programming language.

2. The authenticate API call does not follow the JSON structure as is a custom structure (see below for more information).

3. For the Data API to operate correctly the following changes must be made to *mediaalbumweb.conf*

   ```
   #DATA API OPTIONS
   DATAAPISECRETKEY=xxxxx
   DATAAPIENCRYPTRESULT=1
   ```

   The `DATAAPISECRETKEY` must match the secret used by the script that encrypts and sends the Data API calls. The `DATAAPIENCRYPTRESULT` value, again, must match the value used by the script to decrypt the responses successfully.

<u>Transaction Identifiers</u>

Each API command that is transmitted to Taopix Control Centre will contain one or more transaction identifiers (current transaction and next transaction). The purpose of these identifiers is to prevent the same API command from being executed more than once by a malicious user listening in to the communication.

NOTE

When the Data API is used by an external shopping cart it will already have access to a Taopix Control Centre session. However, this session is only for the interaction between Taopix Designer and the external shopping cart meaning that an authenticated Data API session still needs to be created.

Authenticate API

The *Authenticate API* call is used to create a new authenticated Data API session and must be the first Data API call performed. Before the call is performed a new random value must be passed to the API call that will be the transaction identifier for the next API call.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$nextTransactionRef = random_int(0,32767);

$authenticationResult = clientSideLibrary::authenticate('USERNAME', 'PASSWORD', $nextTransactionRef);
```

The API call accepts the following parameters:

pUsername	The Data API user login configured within Taopix Control Centre
pPassword	The Data API user password configured within Taopix Control Centre
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API

The API call will return an associative array that contains the following values:

error	An error message if the API call failed or empty if it was successful
ref	The Data API session ref if the call was successful

TestSession API

The *TestSession API* call is used to determine that a Taopix shopping cart session is still valid.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::testSession($apiSessionRef, $cartSessionRef, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pSessionToCheckRef	The Taopix shopping cart session reference
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following values:

error	An error message if the API call failed or empty if it was successful
sessionenabled	0=shopping cart session not active 1=shopping cart session still active

EndSession API

The *EndSession API* call is used to inform Taopix Control Centre that a session should be closed. This API call should be used to close a Taopix shopping cart session after triggering a new order and to close the Data API session.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::endSession($apiSessionRef, $cartSessionRef, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pSessionToCloseRef	The session that should be closed
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful

DeleteSession API

The *DeleteSession API* call is used to inform Taopix Control Centre that a session should be deleted immediately.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::deleteSession($apiSessionRef, $cartSessionRef, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pSessionToCloseRef	The session that should be deleted
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful

NOTE
This API call was previously used to inform Taopix that the external shopping cart session had ended. The new *CancelOrder* API should be used instead due to changes that affect Desktop Designer v4.2.0 and later.

Query API

The *Query* API call is used to perform a SQL query within the Taopix database and to return the first record that matches. To perform this API call you must prepare a JSON encoded string in one of the following formats:

1. Simple
This is a simple comparison based SQL Select statement.

```
[
        {
                "action": "SELECT",
                "schema": "taopix database name",
                "ref": "column to compare on",
                "operator": "comparison operator = < > <>",
                "refvalue": "value to compare against",
                "fields":
                        {"column name 1 to return":"", "column name 2 to return":""…},
                "extra": ""
        }
]
```

Example:

```
[
        {
                "action": "SELECT",
                "fields":
                {
                        "ordernumber":"",
                        "orderdate":"",
                        "userid":"",
                        "itemtotalsell":""
                },
                "schema": "ORDERHEADER",
                "ref": "id",
                "operator": "=",
                "refvalue": "500",
                "extra": ""
        }
]
```

2. Advanced
This is an advanced SQL Select statement that uses MySQL bind parameters.

```
[
        {
                "action": "SELECT",
                "extra": "sql statement to perform with ? for MySQL bind parameters",
                "fields":
                {       "column name 1 to return":"", "column name 2 to return":""…},
                "params":
                {
                        "datatype":"list of MySQL bind datatypes",
                        "param1": "param1value",
                        param2…..
                }
        }
]
```

Example:

```
[
        {
                "action": "SELECT",
                "extra": "SELECT ordernumber, orderdate, userid, itemtotalsell FROM ORDERHEADER WHERE id =
?",
                "fields":
                {
                        "ordernumber":"",
                        "orderdate":"",
                        "userid":"",
                        "itemtotalsell":""
                },
                "params":
                {
                        "datatype":"i",
                        "param1": "500"
                }
        }
]
```

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef, 'Query');
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pJSON	The JSON encoded data string
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call
pAction	The DATA API action

The API call will return an associative array that contains at minimum the result of the API call and an entry for each column requested in the ['data']['resultset'] key.
The return array will contain the following values:

error	An error message if the query failed or empty if it was successful
data	Associate array containing the resultset of the SQL command See data array table below

Data array

resultset	Zero based array containing an entry for each column requested by the SQL command
numrows	Total number of returns rows for the SQL command

Insert API

The *Insert* API call is used to perform a SQL insert into the Taopix database. To perform this API call you must prepare a JSON encoded string in one of the following formats:

1. Simple
This is a simple SQL Insert statement.

```
[
        {
                "action": "INSERT",
                "schema": "taopix database name",
                "fields":
                        {
                                "column name 1": "column 1 value",
                                "column name 2": "column 2 value",
                        …},
                "extra": ""
        }
]
```

Example:

```
[
        {
                "action": "INSERT",
                "schema": "ORDERITEMCOMPONENTS",
                "fields":
                {
                        "componentcode": "PAPER.GLOSSYPAPER",
                        "componentpath": "$COVER\\\\WOODCOVER\\\\$PAPER",
                        "componentcategorycode": "PAPER"
                },
                "extra":""
        }
]
```

2. Advanced

This is an advanced SQL Insert statement that uses MySQL bind parameters.

```
[
        {
                "action": "INSERT",
                "extra": "sql statement to perform with ? for MySQL bind parameters",
                "params":
                {
                        "datatype":"list of MySQL bind datatypes",
                        "param1": "param1value",
                        param2…..
                }
        }
]
```

Example:

```
[
        {
                "action": "INSERT",
                "extra":"INSERT INTO ORDERITEMCOMPONENTS
(componentcode,componentpath,componentcategorycode) VALUES (?,?,?)",
                "params":
                {
                        "datatype": "sss",
                        "param1": "PAPER.GLOSSYPAPER",
                        "param2": "$COVER\\\\WOODCOVER\\\\$PAPER",
                        "param3": "PAPER"
                }
        }
]
```

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef, 'Insert');
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pJSON	The JSON encoded data string
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call
pAction	The DATA API action

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the resultset of each SQL command and the total number of affected rows. See data array table below

Data array:

resultset	Zero based array containing the result of each SQL command. See resultset array table below
affectedrows	Total number of inserted rows for all SQL commands

Resultset array:

affected	The number of affected rows for that SQL command
lastinsertid	The record id for the newly inserted record
error	An error message for that SQL command or empty if it was successful

To access the result of the first SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef,
'Insert')

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['resultset'][0]['error'];
    }

    // Check error.
    if ($result === '')
    {
        // First inserted ID.
        $firstInsertedID = $postResult['data']['resultset'][0]['lastinsertid'];
    }
```

Update API

The *Update* API call is used to perform an update to an existing record within the Taopix database. To perform this API call you must prepare a JSON encoded string in one of the following formats:

1. Simple
This is a simple comparison based SQL Update statement.

```
[
        {
                "action": "UPDATE",
                "schema": "taopix database name",
                "ref": "column to compare on",
                "operator": "comparison operator = < > <>",
                "refvalue": "value to compare against",
                "fields":
                        {
                                "column name 1": "column 1 value",
                                "column name 2": "column 2 value",
                        …}
                "extra":""
        }
]
```

Example:

```
[
        {
                "action": "UPDATE",
                "schema": "ORDERHEADER",
                "fields": {
                        "paymentreceived": "1",
                        "paymentreceivedtimestamp":"[currentdatetime]"
                },
                "ref": "id",
                "operator": "=",
                "refvalue": "1",
                "extra": ""
        }
]
```

2. Advanced
This is an advanced SQL Update statement that uses MySQL bind parameters.

```
[
        {
                "action": "UPDATE",
                "extra": "sql statement to perform with ? for MySQL bind parameters",
                "params":
                {
                        "datatype":"list of MySQL bind datatypes",
                        "param1": "param1value",
                        param2…..
                }
        }
]
```

Example:

```
[
        {
                "action": "UPDATE",
                "extra": "UPDATE ORDERHEADER SET paymentreceived = ?, paymentreceivedtimestamp = NOW()
WHERE id = ?",
                "params": {
                        "datatype": "ii",
                        "param1": "1",
                        "param2": "1"
                }
        }
]
```

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef, 'Update');
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pJSON	The JSON encoded data string
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call
pAction	The DATA API action

The API call will return an associative array that contains at minimum the result of the API call and the result of each SQL command.
The return array will contain the following values:

error	An error message if the update failed or empty if it was successful
data	Associate array containing the resultset of each SQL command and the total number of affected rows. See data table below

Data array:

resultset	Zero based array containing the result of each SQL command. See resultset table below
affectedrows	Total number of affected rows for all SQL commands

Resultset array:

affected	The number of affected rows for that SQL command
error	An error message if the SQL command failed or empty if it was successful

Delete API

The *Delete* API call is used to delete an existing record within the Taopix database. To perform this API call you must prepare a JSON encoded string in one of the following formats:

1. Simple
This is a simple comparison based SQL Delete statement.

```
[
        {
                "action": "DELETE",
                "schema": "taopix database name",
                "ref": "column to compare on",
                "operator": "comparison operator = < > <>",
                "refvalue": "value to compare against",
                "extra": ""
        }
]
```

Example:

```
[
        {
                "action": "DELETE",
                "schema": "ORDERITEMCOMPONENTS",
                "ref": "componentcode",
                "operator": "=",
                "refvalue": "COVER.WOODCOVER",
                "extra":""
        }
]
```

2. Advanced

This is an advanced SQL Delete statement that uses MySQL bind parameters.

```
[
        {
                "action": "DELETE",
                "extra": "sql statement to perform with ? for MySQL bind parameters",
                "params":
                {
                        "datatype":"list of MySQL bind datatypes",
                        "param1": "param1value",
                        param2…..
                }
        }
]
```

Example:

```
[
        {
                "action": "DELETE",
                "extra":"DELETE FROM ORDERITEMCOMPONENTS WHERE componentcode = ?",
                "params": {
                        "datatype": "s",
                        "param1": "COVER.WOODCOVER"
                }
        }
]
```

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef, 'Delete');
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pJSON	The JSON encoded data string
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call
pAction	The DATA API action

The API call will return an associative array that contains at minimum the result of the API call. The return array will contain the following values:

error	An error message if the delete failed or empty if it was successful
data	Associate array containing the resultset of each SQL command and the total number of deleted rows. See data table below

Data array:

resultset	Zero based array containing the result of each SQL command. See resultset table below
affectedrows	Total number of deleted rows for all SQL commands

Resultset array:

affected	The number of deleted rows for that SQL command
error	An error message if the SQL command failed or empty if it was successful

Execute API

The *Execute* API call is used to execute arbitrary SQL commands against the database. To perform this API call you must prepare a JSON encoded string in the following format:

```
[
        {
                "action": "EXECUTE",
                "query": "command to perform",
        }
]
```

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef, 'Execute');
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pJSON	The JSON encoded data string
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call
pAction	The DATA API action

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays inside ['data']['resultset'] to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the query failed or empty if it was successful
data	Associate array containing the resultset of each SQL. See data table below

Data array:

resultset	Zero based array containing the result of each SQL command. See resultset table below

Resultset array:

error	An error message if the SQL command failed or empty if it was successful

To access the result of the first SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::apiCall($apiSessionRef, $jsonString, $nextTransactionRef,
'Execute')

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['resultset'][0]['error'];
    }

    // Check error.
    if ($result === '')
    {
        // First inserted ID.
        $firstInsertedID = $postResult['data']['resultset'][0]['lastinsertid'];
    }
```

TriggerOrder API

The *TriggerOrder API* call is used to inform Taopix Control Centre that a new order has been successfully inserted into the Taopix database and must be performed before ending the session.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
    $postResult = clientSideLibrary::triggerOrder($apiSessionRef, $cartSessionRef, $orderHeaderID,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderSessionID	The Taopix shopping cart session reference If the order was created via Taopix Online using the Taopix E-Commerce Integration API or the Taopix External Checkout API this value should be set to 'NOORDERSESSIONREF'.
pOrderID	The Taopix database order header record id returned after inserting the order header record
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful

CancelOrder API

The *CancelOrder API* call is used to inform Taopix Control Centre that an external shopping cart session has been aborted and is no longer valid.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
    $postResult = clientSideLibrary::cancelSession($apiSessionRef, $cartSessionRef, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pSessionToCloseRef	The session that should be cancelled
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful

CheckDataAvailable API

The *CheckDataAvailable API* call is used to query Taopix Control Centre to discover if the relevant order data for an order item is available. If the order has been cleaned up or is the process of being cleaned up the data will be flagged as unavailable.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::checkDataAvailable($apiSessionRef, $orderItemID, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderItemId	The ID of the order item you wish to query.
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful
dataavailable	True if the data is still available. False is the order item has been cleaned up or is in the process of being cleaned up.

getProjectOrderData API

The *getProjectOrderData API* call is used to query Taopix Control Centre to retrieve project order data required to insert an order into the database.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
    $postResult = clientSideLibrary::getProjectOrderData($apiSessionRef, $orderNumber, $projectRefArray,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderNumber	The Order number with any relevant formatting you wish to assign the project data to.
pProjectRefArray	An array of Taopix project refs that are to be inserted as part of the order.
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call.

The API call will return an associative array that contains the following values:

error	One of the following: 0 – Indicates that the call has returned order data for the project refs provided. 1 – Indicates that there was a database error. 2 – Indicates that there were no project refs provided. 3 – Indicates that there were no projects found based on the project refs provided. 4 – Indicates that the call has returned order data for only some of the projects based of the project refs provided.
errormessage	An error message that corresponds to the above error codes.
missingprojectrefs	An array of projects refs that were not found.
orderdata	An associative array that contains the order data needed to insert the order into the database. headerarray – Contains the order header associative array as described under Order Header in the Inserting An Order Into Taopix Section. cartarray – An array of order items that are to be inserted as part of the order. For order item the associative array includes external assets, pictures and calendar customisations. The structure of these associative arrays are described in the Inserting An Order Into Taopix Section. shippingdata – Contains the shipping data associative array as described under Order Shipping in the Inserting An Order Into Taopix Section.

INSERTING AN ORDER INTO TAOPIX

When the customer confirms their order within the external shopping cart it must use the Taopix Data API to insert an order into the Taopix database.

Each order within the Taopix database consists of the following:
a) An optional matching customer USER record
b) One ORDERHEADER record
c) One or more ORDERITEMS records
d) Zero or more ORDERITEMCOMPONENTS records
e) One ORDERSHIPPING record

Customer Record

Depending on the requirements of the integration there may or may not be a requirement to link the order to a customer account within the Taopix database.

If a *userid* with a value greater than zero is supplied during the initialisation step a link can be made by simply using this value in the order header.

If a *userid* with a value of zero is supplied during the initialisation step it is up to the integrator to determine if they wish to link the order to a Taopix customer account. A link is required if you wish to use the order related features within Taopix Control Centre (re-ordering, social sharing etc…). If a link is not required then the value of zero can be used in the order header. If a link is required then either an existing Taopix user account must be located or one created. This can be achieved by using the Taopix Data API Query API and Insert API described earlier in this document and the information relating to inserting and updating user accounts found later in this document.

NOTE
It is assumed here that if a *userid* with a value greater than zero is supplied during the initialisation step, the external cart has automatically authenticated the customer with an account in the external system that matches the Taopix account. If this cannot be achieved the *userid* supplied should be ignored. However, it is strongly recommended that the customer is automatically authenticated otherwise this could result in a mismatch on who owns an order between the systems.

insertOrder API

The *insertOrder API* call is used to insert an entire order into the database. This call attempts to simplify inserting the order and will insert the order header, order items, order item components, order shipping as described below. If all of these inserts are successful, then the trigger order as described below will also be executed as part of this API call.

If the `clientSideLibrary.php` class is being used then the following code example will show how to call the API:

```
$postResult = clientSideLibrary::insertOrder($apiSessionRef, $orderData, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderData	An associative array that contains the order data needed to insert the order into the database. This associative array should be in the same format as the array returned from the getProjectOrderData API.
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call.

The API call will return an associative array that contains the following value:

error	An error message if the API call failed or empty if it was successful

Order Header

The order header contains information on the overall order such as the order date and order number along with billing, payment and summary information and must be the first part of an order that is inserted into the Taopix database.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderHeader($apiSessionRef, $orderHeader,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderHeader	The PHP associative array that contains the order header data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderHeader($apiSessionRef, $orderHeader,
$nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

$

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
ownercode	varchar(50)	Y	Received from External Script
groupcode	varchar(50)	Y	Received from External Script
groupdata	varchar(50)	Y	Received from External Script
orderdate	datetime	Y	Order date and time in MySQL date / time format
ordernumber	varchar(50)	Y	Order number with any relevant formatting
origorderid	integer	Y	Received from External Script
origordernumber	varchar(50)	Y	Received from External Script
webbrandcode	varchar(50)	Y	Received from External Script
uuid	varchar(200)	Y	Received from External Script
browserlanguagecode	varchar(10)	Y	Code of the language used by the customer while placing their order. This must match a language code supported by Taopix
userid	integer	Y	The id within the Taopix database for the user record this order belongs to. If a *userid* with a number greater than zero was supplied during the initialisation step then this number should be used. If using only Taopix desktop, a userid of 0 can be used to not link it to an account. If using Taopix Online 0 cannot be used as it always requires a userid and the userid must exist in the Taopix Control Centre database. This could be a generic userid setup for the purposes of the API.
currencycode	varchar(20)	Y	The code of the currency used in this order. This must match a currency code defined within Taopix Control Centre.
currencyname	varchar(1024)	Y	The name of the currency used within the order in Taopix multi-lingual string format
currencyisonumber	varchar(3)	Y	The ISO number of the currency used in the order
currencysymbol	varchar(5)	Y	The currency symbol
currencysymbolatfront	integer	Y	Used to format currency values. 0=currency symbol is at the end 1=currency symbol at the front
currencydecimalplaces	integer	Y	The number of decimal places used by the currency (used to format currency values)
currencyexchangerate	decimal(10,4)	Y	The currency exchange rate or 1
basecurrencycode	varchar(20)	Y	The base currency code if an exchange rate has been applied otherwise the same as the currency code.
ordertotalitemsell	decimal(10,2)	Y	The total sell price of the line items
ordertotalitemtax	decimal(10,2)	Y	The total tax applied to the line items
shippingtotalsellbeforediscount	decimal(10,2)	Y	The total shipping sell price before any shipping discount has been applied
shippingtotalsell	decimal(10,2)	Y	The total shipping sell price

ordertotalshippingtax	decimal(10,2)	Y	The total amount of tax applied to shipping
ordertotal	decimal(10,2)	Y	The final order total
paymentmethodcode	varchar(20)	Y	The code of the payment method used on this order. This must match one of the following payment methods defined within Taopix Control Centre: ACCOUNT COD CARD CHEQUE PAYPAL
paymentmethodname	varchar(1024)	Y	The name of the payment method in Taopix multi-lingual format.
ccilogid	integer	Y	The id of the record within the ccilog table that contains information on the payment gateway transaction otherwise 0 if not required.
itemcount	integer	Y	The number of line items within the order.
billingcustomeraccountcode	varchar(50)	N	Customer's billing account
billingcustomername	varchar(200)	N	Customer's billing company name
billingcustomeraddress1	varchar(200)	N	Customer's billing address line 1
billingcustomeraddress2	varchar(200)	N	Customer's billing address line 2
billingcustomeraddress3	varchar(200)	N	Customer's billing address line 3
billingcustomeraddress4	varchar(200)	N	Customer's billing address line
billingcustomercity	varchar(200)	N	Customer's billing city
billingcustomercounty	varchar(200)	N	Customer's billing county
billingcustomerstate	varchar(200)	N	Customer's billing state
billingcustomerregioncode	varchar(20)	N	Customer's billing region code. This must either be empty or match an address region code within the Taopix database
billingcustomerregion	varchar(10)	N	Customer's billing region field. This identifies which address field contains the region and can either be empty, STATE or COUNTY
billingcustomerpostcode	varchar(200)	N	Customer's billing postcode / zipcode
billingcustomercountrycode	varchar(2)	N	Customer's billing country code. This must match a country code within the Taopix database
billingcustomercountryname	varchar(64)	N	Customer's billing country name
billingcustomertelephonenumber	varchar(50)	N	Customer's billing telephone number
billingcustomeremailaddress	varchar(50)	N	Customer's billing email address
billingcontactfirstname	varchar(200)	N	Customer's billing contact first name
billingcontactlastname	varchar(200)	N	Customer's billing contact last name
useremaildestination	integer	N	
ordertotalitemcost	decimal(10,2)	N	The total cost of all order lines
ordertotalshippingcost	decimal(10,2)	N	The total shipping cost for all order lines

ordertotalshippingweight	decimal(10,4)	N	The total shipping weight for all order lines
ordertotalcost	decimal(10,2)	N	The total cost of the order
totalsellbeforediscount	decimal(10,2)	N	The total sell price before any discount has been applied
ordertotaltaxbeforediscount	decimal(10,2)	N	The total tax amount before any discount has been applied
ordertotaldiscount	decimal(10,2)	N	The total discount that has been applied to the order
totalsell	decimal(10,2)	N	The order total sell price without tax applied
ordertotaltax	decimal(10,2)	N	The total amount of tax applied to the order
paymentgatewaycode	varchar(20)	N	The code of the payment gateway used on this order.
paymentreceived	integer	N	Has payment been received 0=no 1=yes
paymentreceivedtimestamp	datetime	N	Time stamp of when payment was marked as received in MySQL date / time format
paymentreceiveddate	datetime	N	The date and time the payment was received in MySQL date / time format
paymentreceiveduserid	integer	N	The record id within the Taopix database for the user that marked this payment as received (or -1 if set automatically)
voucherpromotioncode	varchar(50)	N	The voucher promotion code
voucherpromotionname	varchar(100)	N	The voucher promotion name
vouchercode	varchar(50)	N	The voucher code
vouchername	varchar(1024)	N	The voucher name in Taopix multi-lingual string format
voucherdiscountsection	varchar(50)	N	The section of the order the discount is applied to. This should either be empty or one of the following: PRODUCT SHIPPING TOTAL
voucherdiscounttype	varchar(50)	N	The discount type applied to the order. This should either be empty or one of the following codes: FOC (free of charge) VALUE (value off) PERCENT (percentage off) BOGOF (buy qty get one free) BOGPOFF (buy qty get percentage off) BOGVOFF (buy qty get value off) VALUESET (fixed value)

voucherdiscountvalue	decimal(10,2)	N	The value of the voucher in the discount type format
totalbeforediscount	decimal(10,2)	N	The order total before any discount has been applied
shippingrequiresdelivery	integer	N	Specifies if the shipping of the order requires delivery. This relates to how payment methods can be filtered within Taopix Control Centre based on the shipping method. 0=no 1=yes
showpriceswithtax	integer	N	Specifies if prices were displayed with tax included: 0=no 1=yes
showzerotax	integer	N	Specifies if tax when the rate is 0% is displayed 0=no 1=yes
showtaxbreakdown	integer	N	Specifies if the breakdown of tax should be displayed 0=no 1=yes
showalwaystaxtotal	integer	N	Specifies if the tax total should be displayed 0=no 1=yes

Order Items

After successfully inserting an order header the next step is to insert an order item. An order item record contains information on the Taopix product that has been purchased.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderItem($apiSessionRef, $orderItem,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderItem	The PHP associative array that contains the order item data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the results of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderItem($apiSessionRef, $orderItem,
$nextTransactionRef);


    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

orderid	integer	Y	The record id of the previously inserted order header record
projectref	varchar(300)	Y	Received from External Script
projectreforig	varchar(300)	Y	Received from External Script
projectname	varchar(200)	Y	Received from External Script
origorderid	integer	Y	Received from External Script
origorderitemid	integer	Y	Received from External Script
source	integer	Y	Received from External Script
shareid	integer	Y	Received from External Script
itemnumber	integer	Y	The line item number within the order starting with 1.
uploadgroupcode	varchar(50)	Y	Received from External Script
uploadorderid	integer	Y	Received from External Script
uploadordernumber	varchar(50)	Y	Received from External Script
uploadorderitemid	integer	Y	Received from External Script
uploadbatchref	varchar(200)	Y	Received from External Script
uploadref	varchar(200)	Y	Received from External Script
uploadappversion	varchar(20)	Y	Received from External Script
uploadappplatform	varchar(20)	Y	Received from External Script
uploadappcputype	varchar(20)	Y	Received from External Script
uploadapposversion	varchar(50)	Y	Received from External Script
collectioncode	varchar(50)	Y	Received from External Script
collectionname	varchar(1024)	Y	Received from External Script
productcode	varchar(50)	Y	Received from External Script
productname	varchar(1024)	Y	Received from External Script
pagecount	integer	Y	Received from External Script
taxcode	varchar(50)	Y	The code of the tax rate that has been applied to the line item. This must match a tax rate code defined within Taopix Control Centre
taxname	varchar(1024)	Y	The name of the tax rate that has been applied to the line item in Taopix multi-lingual string format
taxrate	decimal(10,4)	Y	The tax rate applied to the line item
qty	integer	Y	The item / line quantity
producttotalsell	decimal(10,2)	Y	The product total sell price based on the quantity ordered. Depending on how pricing is implemented this could be unitprice x qty
producttotaltax	decimal(10,2)	Y	The product total tax amount based on the quantity ordered
subtotal	decimal(10,2)	Y	The line sub-total before any discount has been applied
discountvalue	decimal(10,2)	Y	The total amount of discount applied to the line
totalsell	decimal(10,2)	Y	The line total sell based on the quantity ordered
taxtotal	decimal(10,2)	Y	The line total tax based on the quantity ordered
producttype	integer	Y	Received from External Script
productpageformat	integer	Y	Received from External Script
productspreadpageformat	integer	Y	Received from External Script
productcover1format	integer	Y	Received from External Script
productcover2format	integer	Y	Received from External Script
productoutputformat	integer	Y	Received from External Script
productheight	decimal(10,6)	Y	Received from External Script
productwidth	decimal(10,6)	Y	Received from External Script
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
orderwebversion	varchar(20)	N	A string indicating the version number of the integration
productinfo	varchar(1024)	N	Additional product description information in Taopix multi-lingual string format

assetid	integer	N	The id within the Taopix database for the product's preview picture asset
productunitcost	decimal(10,2)	N	Received from External Script
productunitsell	decimal(10,2)	N	The unit sell price of the product. Depending on how pricing has been implemented this could be: product total sell / qty
productskucode	varchar(50)	N	Received from External Script
productdefaultpagecount	integer	N	Received from External Script
productunitweight	decimal(10,4)	N	Received from External Script
producttotalweight	decimal(10,4)	N	Product total weight based on the qty ordered
producttotalcost	decimal(10,2)	N	Product total cost based on the qty ordered
totalcost	decimal(10,2)	N	The line total cost based on the qty ordered
totalweight	decimal(10,4)	N	The line total weight based on the qty ordered
voucherapplied	integer	N	Was the voucher applied to this line 0=no 1=yes

Order Item Components

After successfully inserting an order item the next step is to insert any order item components. An order item component record is an optional record that contains information on any additional add-ons to the product such as a cover, a paper stock, a gift box etc…

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderItemComponent($apiSessionRef, $orderItemComponent,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pComponent	The PHP associative array that contains the order item component data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderItemComponent($apiSessionRef, $orderItemComponent,
$nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
orderitemid	integer	Y	The record id of the order item that this component belongs to
parentcomponentid	integer	Y	The record id of the parent order item component if this is a sub-component otherwise 0 if this is not a sub-component.
componentcode	varchar(152)	Y	The combination of the component category code and component local code separated with a full stop
componentlocalcode	varchar(50)	Y	The component code. This must match the code of a component created within the Taopix database
componentdefaultcode	varchar(152)	Y	If the component belongs to a list this contains the code of the component that was the default option within the list
componentname	varchar(1024)	Y	The name of the component in Taopix multi-lingual string format
componentpath	varchar(1024)	Y	The path to the component. Parent components use the format: $*[componentcategorycode]*\ Sub-components use the format: $*[componentcategorycode]**[componentcode]*\ $*[subcomponentcategorycode]*\
componentcategorycode	varchar(50)	Y	The component category code. This must match the code of a component category created within the Taopix database
componentcategoryname	varchar(1024)	Y	The name of the component category in Taopix multi-lingual string format
sortorder	integer	Y	The order in which the component appears within the order line
pricingmodel	integer	Y	The pricing model used for the component: 3=Per Line Qty 5=Per Line Qty x Per Side Qty 7=Per Line Qty x Component Qty 8=Per Line Qty x Component Qty x Per Side Qty
islist	integer	Y	0=checkbox (on/off) component 1=list component
itemselected	integer	Y	If the component is a checkbox component this indicates if the component was selected. 0=not selected 1=selected
componenttotalsell	decimal(10,2)	Y	The total sell price of the component based on the order item quantity. Depending on how pricing is implemented this could be: unitsell x order item qty
componenttotaltax	decimal(10,2)	Y	The total amount of tax applied to the component
quantity	integer	Y	Must be set to 1 unless the pricing model supports the component quantity.
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
skucode	varchar(50)	N	Component SKU code
componentunitcost	decimal(10,2)	N	The unit cost price of the component
componentunitsell	decimal(10,2)	N	The unit sell price of the component. Depending on how pricing has been implemented this could be: component total sell / order item qty

componenttotalcost	decimal(10,2)	N	The total cost of the component
componentdescription	varchar(1024)	N	The description of the component in Taopix multi-lingual string format
componentselectioncount	integer	N	If the component is a list component the number of items in the list that could have been chosen
componentunitcost	decimal(10,2)	N	The unit cost of the component
componentunitweight	decimal(10,4)	N	The unit weight of the component
componenttotalweight	decimal(10,4)	N	The total weight of the component for the order item

Inserting Sub-Components

API version 7.0 onwards supports the ability to insert sub-components as well as components. Inserting a sub-component involves first inserting the parent component to retrieve it's ID and then inserting the sub-component with the correct data including the parent component ID and path. Please refer to the updated sample code for more information.

Inserting External Assets

Orders generated by Taopix Designer v3.2.0 onwards may contain a list of Taopix Library Link external assets that have been used within a project. This data can be used by the external shopping cart to charge the customer for the assets as well as being stored in the Taopix database for reporting purposes. Storing the external assets within the Taopix database involves inserting a special type of order item component that has the code TAOPIXASSET.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderItemExternalAsset($apiSessionRef, $externalAsset,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pExternalAsset	The PHP associative array that contains the external asset component data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderItemExternalAsset($apiSessionRef, $externalAsset,
$nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
orderitemid	integer	Y	The record id of the order item that this component belongs to
externalassetservicecode	varchar(50)	Y	Received from External Script
externalassetservicename	varchar(1024)	Y	Received from External Script
externalassetpricetype	integer	Y	Received from External Script
externalassetid	varchar(50)	Y	Received from External Script
externalassetname	varchar(1024)	Y	Received from External Script
sortorder	integer	Y	The order in which the component appears within the order line
externalassetunitcost	decimal(10,2)	Y	Received from External Script
externalassetunitsell	decimal(10,2)	Y	Received from External Script
externalassetexpirydate	datetime	Y	Received from External Script
externalassetpageref	integer	Y	Received from External Script
externalassetpagenumber	integer	Y	Received from External Script
externalassetpagename	varchar(1024)	Y	Received from External Script
externalassetboxref	integer	Y	Received from External Script
componentunitcost	decimal(10,2)	Y	Received from External Script
componentunitsell	decimal(10,2)	Y	Received from External Script
componenttotalsell	decimal(10,2)	Y	The total sell price of the component based on the order item quantity. Depending on how pricing is implemented this could be: unitsell x order item qty
componenttotalcost	decimal(10,2)	Y	The total cost of the component
componentunitcost	decimal(10,2)	Y	The unit cost of the component
componenttotaltax	decimal(10,2)	Y	The total amount of tax applied to the component
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)

NOTE
Both the Taopix shopping cart and the external shopping cart code insert external asset order item component records before regular order item component records. It is recommended that you follow this approach when modifying your integration to support external assets.

Inserting Pictures (Single Prints and Photo Prints)

API version 7.0 onwards exposes the picture data used by the Taopix Single Prints module (Taopix Desktop) and Taopix Photo Prints Module (Taopix Online) to the external shopping cart. This data can be used by the external shopping cart to charge the customer for their prints as well as being stored in the Taopix database as components for production and reporting purposes.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderItemPicture($apiSessionRef, $picture,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pPicture	The PHP associative array that contains the picture component data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderItemPicture($apiSessionRef, $picture,
$nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
orderitemid	integer	Y	The record id of the order item that this component belongs to
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
parentcomponentid	integer	Y	The record id of the parent order item component if this is a sub-component otherwise 0 if this is not a sub-component.
setid	integer	Y	Received from External Script
setname	varchar(200)	Y	Received from External Script
externalassetservicecode	varchar(50)	Y	Received from External Script
externalassetservicename	varchar(1024)	Y	Received from External Script
externalassetpricetype	integer	Y	Received from External Script
componentcode	varchar(152)	Y	Received from External Script
componentlocalcode	varchar(50)	Y	Received from External Script
externalassetid	varchar(50)	Y	Received from External Script
componentname	varchar(1024)	Y	Received from External Script
componentpath	varchar(1024)	Y	Received from External Script
componentcategorycode	varchar(50)	Y	Received from External Script
sortorder	integer	Y	Received from External Script
quantity	integer	Y	Received from External Script
componentunitcost	decimal(10,2)	N	The unit cost price of the component. The external script provides this but it is recommended that the value from the external cart is used
componentunitsell	decimal(10,2)	N	The unit sell price of the component. The external script provides this but it is recommended that the value from the external cart is used
externalassetunitcost	decimal(10,2)	Y	Received from External Script
externalassetunitsell	decimal(10,2)	Y	Received from External Script
componentunitweight	decimal(10,4)	N	The unit weight of the component. The external script provides this but it is recommended that the value from the external cart is used
componenttotalcost	decimal(10,2)	N	The total cost of the component
componenttotalsell	decimal(10,2)	Y	The total sell price of the component
componenttotalweight	decimal(10,4)	N	The total weight of the component for the order item
componenttotaltax	decimal(10,2)	Y	The total amount of tax applied to the component
externalassetpageref	integer	Y	Received from External Script
externalassetpagenumber	integer	Y	Received from External Script
externalassetpagename	varchar(1024)	Y	Received from External Script
externalassetboxref	integer	Y	Received from External Script

NOTE

Both the Taopix shopping cart and the external shopping cart code insert single print/photo print order item component records after external asset order item components and before regular order item component records. It is recommended that you follow this approach when modifying your integration to support external assets.

Inserting Calendar Customisation

API version 8.0 onwards exposes the calendar customisation used by the Enhanced Calendar module introduced in Taopix v2016r1 to the external shopping cart. This data can be used by the external shopping cart to charge the customer for any calendar customisation components within their project that are classes as chargeable as well as being stored in the Taopix database as components for production and reporting purposes.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary:: insertOrderItemCalendarComponent($apiSessionRef,
$calendarcustomisation, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pCalendarComponent	The PHP associative array that contains the calendar component data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary:: insertOrderItemCalendarComponent($apiSessionRef,
$calendarcustomisation, $nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
orderitemid	integer	Y	The record id of the order item that this component belongs to
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
componentcode	varchar(152)	Y	Received from External Script
componentlocalcode	varchar(50)	Y	The component code. This must match the code of a component created within the Taopix database
skucode	varchar(50)	N	Received from External Script
componentname	varchar(1024)	Y	Received from External Script
componentpath	varchar(1024)	Y	The path to the component. Parent components use the format: $*[componentcategorycode]*\ Sub-components use the format: $*[componentcategorycode]**[componentcode]*\ $*[subcomponentcategorycode]*\
quantity	integer	Y	Received from External Script
componentunitcost	decimal(10,2)	N	The unit cost price of the component. The external script provides this but it is recommended that the value from the external cart is used
componentunitsell	decimal(10,2)	N	The unit sell price of the component. The external script provides this but it is recommended that the value from the external cart is used
componenttotalcost	decimal(10,2)	N	The total cost of the component
componenttotalsell	decimal(10,2)	Y	The total sell price of the component
componenttotaltax	decimal(10,2)	Y	The total amount of tax applied to the component

Calendar customisations can be priced one of three different ways; *DATE*, *EVENTSET* and *ANY*.

DATE pricing will be applied to an order for each date which has been customised by the end user. This means adding event text, a picture or a colour to a date.

EVENTSET pricing will be applied to an order for each predefined event set which an end user has switched on.

The *ANY* pricing will <u>only</u> be applied if <u>both</u> the *DATE* and *EVENTSET* components have not been configured on the component tree or are inactive at the time of ordering. The quantity of this component will be the sum of the *DATE* and *EVENTSET* quantities. Storing just the *ANY* component in the database poses a problem for reordering. If at the time of reordering the calendar customisation components which are configured for the product have changed, which means the ANY component has been removed and both *DATE* and *EVENTSET* component have been added, there is no way of knowing what each component's quantity is. The solution to this problem is to store both the *DATE* and *EVENTSET* components without their total values when inserting an *ANY* component. This means that when reordering the individual quantities have been preserved and can be used to calculate the prices of these components. This table helps explain this:

Components configured on the product	What needs storing on the orderitemscomponents table		
	Component	**Qty**	**Pricing Data?**
ANY	ANY	DATE Qty + EVENTSET Qty	Yes
	DATE	DATE Qty	No
	EVENTSET	EVENTSET Qty	No
DATE	DATE	DATE Qty	Yes
	EVENTSET	Zero	No
EVENTSET	EVENTSET	EVENTSET Qty	Yes

	DATE	Zero	No
DATE and EVENTSET	DATE	DATE Qty	Yes
	EVENTSET	EVENTSET Qty	Yes
DATE and ANY	DATE	DATE Qty	Yes
EVENTSET and ANY	EVENTSET	EVENTSET Qty	Yes
ANY, DATE and EVENTSET	DATE	DATE Qty	Yes
	EVENTSET	EVENTSET Qty	Yes

The *ANY* component will only appear in the calendarcustomisation array if it is being used. The *DATE* and *EVENTSET* components will always appear in the array whether they have been used or not. If they have been used the pricing information for the component will be populated.

IMPORTANT
The *ANY* component should only be configured for a product on its own and not alongside *DATE* or *EVENTSET*.

NOTE
Both the Taopix shopping cart and the external shopping cart code insert calendar customisation order item component records after Single Print/Photo Print order item components and before regular order item component records. It is recommended that you follow this approach when modifying your integration to support calendar customisation.

Inserting Retro Prints Components

API version 12.0.1 onwards allows you to insert components required for use with Retro Prints products which was introduced in Taopix v2021r2. These components are used to know how many copies of a page ("print") to produce. It is not necessary to call this function using the insertOrder function to insert an order as that function will take care of inserting the components automatically.

Note: To be able to use Retro Prints and this function, please contact Taopix Support.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary:: insertOrderRetroPrintsComponent($apiSessionRef,
$retroPrintComponentArray, $nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pRetroPrintComponent	A PHP associative array that contains the Retro Prints component data. See the following table
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
orderitemid	integer	Y	The record id of the order item that this component belongs to
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
pagenumber	integer	Y	Page number that the component relates to
quantity	integer	Y	The number of prints to be made for the page ("print")

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

data array:

affectedrows	The total number of affected rows for the command
resultset	Numbered array containing details for each query. In this case there will only be 1 result. See resultset table below

resultset array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary:: insertOrderItemCalendarComponent($apiSessionRef,
$retroPrintComponetArray, $nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

Order Shipping

After successfully inserting an order item and any order item components the final step is to insert the order shipping record. The order shipping record contains information on the shipping address.

If the `clientSideLibrary.php` class is being used a function is available that accepts a PHP array, converts it to JSON and executes the API call.

```
    $postResult = clientSideLibrary::insertOrderShipping($apiSessionRef, $orderShipping,
$nextTransactionRef);
```

The API call accepts the following parameters:

pAPISessionRef	The Data API session reference returned from the Authenticate API call
pOrderShipping	The PHP associative array that contains the order shipping data
pNextTransactionRef	The transaction identifier for the next transaction that will be submitted to the API. This parameter is passed by reference and will be updated to contain a new transaction identifier for the next API call

The API call will return an associative array that contains at minimum the result of the API call. If the *error* value is empty then the result of the actual SQL insert command should be checked. The SQL command result is held in a list of associative arrays to allow future support for multiple SQL commands in a single API call.

The return array will contain the following values:

error	An error message if the insert failed or empty if it was successful
data	Associate array containing the result of the command and the total number of affected rows. See data array table below

Data array:

affected	The number of affected rows for the command
lastinsertid	The record id for the newly inserted record
error	An error message for the command or empty if it was successful

To access the result of the SQL command you would perform something similar to the following:

```
    $postResult = clientSideLibrary::insertOrderShipping($apiSessionRef, $orderShipping,
$nextTransactionRef);

    // Retrieve the execution error.
    $result = $postResult['error'];

    // Check error.
    if ($result === '')
    {
        // Retrieve the query execution error from the database command.
        $result = $postResult['data']['error'];
    }

    // Check error.
    if ($result === '')
    {
        // Inserted ID.
        $insertedID = $postResult['data']['lastinsertid'];
    }
```

The PHP associative array passed to the function must contain the following keys and their required values. The *Type, Mandatory* and *Description* columns listed below are for reference purposes only:

Name	Type	Mandatory	Description
orderid	integer	Y	The record id of the previously inserted order header record
shippingmethodcode	varchar(50)	Y	The code of the shipping method that has been used. This must match a shipping method code defined within Taopix Control Centre
shippingmethodname	varchar(1024)	Y	The name of the shipping method in Taopix multi-lingual string format
shippingratesell	decimal(10,2)	Y	The shipping rate sell price
shippingratediscountvalue	decimal(10,2)	Y	The amount of discount applied to the shipping rate
shippingratetaxcode	varchar(50)	Y	The code of the tax rate that has been applied to the shipping. This must match a tax rate code defined within Taopix Control Centre
shippingratetaxname	varchar(1024)	Y	The name of the tax rate that has been applied to the shipping in Taopix multi-lingual string format
shippingratetaxrate	decimal(10,2)	N	The tax rate applied to the line item
shippingratecalctax	integer	N	Specifies if tax was calculated on the shipping rather than on the order total 0=no 1=yes
shippingratetaxtotal	decimal(10,2)	Y	The total amount of tax applied to the shipping
shippingratetotalsell	decimal(10,4)	Y	The shipping rate total
userid	integer	Y	The id within the Taopix database for the user record this order belongs to (use 0 if no matching user account)
shippingcustomername	varchar(200)	N	Customer's shipping company name
shippingcustomeraddress1	varchar(200)	N	Customer's shipping address line 1
shippingcustomeraddress2	varchar(200)	N	Customer's shipping address line 2
shippingcustomeraddress3	varchar(200)	N	Customer's shipping address line 3
shippingcustomeraddress4	varchar(200)	N	Customer's shipping address line
shippingcustomercity	varchar(200)	N	Customer's shipping city
shippingcustomercounty	varchar(200)	N	Customer's shipping county
shippingcustomerstate	varchar(200)	N	Customer's shipping state
shippingcustomerregioncode	varchar(20)	N	Customer's shipping region code. This must either be empty or match an address region code within the Taopix database
shippingcustomerregion	varchar(10)	N	Customer's shipping

			region field. This identifies which address field contains the region and can either be empty, STATE or COUNTY
shippingcustomerpostcode	varchar(200)	N	Customer's shipping postcode / zipcode
shippingcustomercountrycode	varchar(2)	N	Customer's shipping country code. This must match a country code within the Taopix database
shippingcustomercountryname	varchar(64)	N	Customer's shipping country name
shippingcustomertelephonenumber	varchar(50)	N	Customer's shipping telephone number
shippingcustomeremailaddress	varchar(50)	N	Customer's shipping email address
shippingcontactfirstname	varchar(200)	N	Customer's shipping contact first name
shippingcontactlastname	varchar(200)	N	Customer's shipping contact last name
shippingratecode	varchar(50)	N	The code of the shipping rate that has been used or empty if not known.
shippingrateinfo	varchar(1024)	N	The additional descriptive information for the shipping rate
shippingratecost	decimal(10,2)	N	The cost of the shipping

Triggering A New Order

After all data for an order (order header, order items, order item components, order shipping information) has been successfully inserted into the Taopix database the *TriggerOrder* and *EndSession* API calls must be used to finish the interaction with Taopix Control Centre.

Inserting A New Customer Account

If a new user account is required the following table lists the data that can be provided to the Taopix Data API Insert API command. Successful calls to the Insert API will result in the new record being inserted and the new record id being returned so that it can be referenced elsewhere.

webbrandcode	varchar(50)	M	The Taopix Control Centre Brand code that was supplied during the initialisation process.
login	varchar(50)	M	The login name for the customer account which must be unique for the brand.
password	varchar(50)	O	An MD5 Hash of the account password if you wish the customer to be able to sign-in to Taopix to view their account and the customer accounts are not being controlled via an external script.
customer	integer	M	Must be set to 1
groupcode	varchar(50)	M	The Taopix License Key code that was supplied during the initialisation process.
accountcode	varchar(50)	O	The account code for the user account (this could be used to match customer accounts between different systems)
companyname	varchar(200)	A	Company name
address1	varchar(200)	A	Address line 1
address2	varchar(200)	A	Address line 2
address3	varchar(200)	A	Address line 3
address4	varchar(200)	A	Address line 4
city	varchar(200)	A	City
county	varchar(50)	A	County
state	varchar(200)	A	State
regioncode	varchar(20)	A	Region code
region	varchar(10)	A	Region field code
postcode	varchar(200)	A	Postcode / Zipcode
countrycode	varchar(10)	M	The *iso2* country code that matches a value within the Taopix *COUNTRIES* database table.
countryname	varchar(50)	M	The name of the country that matches the *iso2* country code.
telephonenumber	varchar(50)	M	Telephone number
emailaddress	varchar(50)	M	Email address
contactfirstname	varchar(200)	M	Contact first name
contactlastname	varchar(200)	M	Contact last name
registeredtaxnumbertype	integer	A	The tax number type for the customer. Values greater than 0 are currently only supported for addresses in Brazil. 0 = Not applicable 1 = Personal 2 = Corporate
registeredtaxnumber	varchar(50)	A	The tax number in the format specified by *registeredtaxnumbertype* when *registeredtaxnumbertype* is not 0.
usedefaultpaymentmethods	integer	M	0 = Use custom set of payment methods 1 = Use default payment methods
paymentmethods	varchar(100)	C	A comma separated list of Taopix payment method codes when using *usedefaultpaymentmethods* is 0.
taxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom product tax rate to the customer account or empty if Taopix should determine the product tax rate.
shippingtaxcode	varchar(20)	O	A valid Taopix Control Centre tax rate code if applying a custom shipping tax rate to the customer account or empty if Taopix should determine the shipping tax rate.
defaultaddresscontrol	integer	C	0 = The address / email settings are customised for this user account. 1 = The address / email settings are controlled by

			the license key.
uselicensekeyforshippingaddress	integer	C	0 = The user account address is used for the shipping address. 1 = The address held in the license key is used for the shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingaddress	integer	C	0 = The shipping address cannot be modified. 1 = The shipping address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifyshippingcontactdetails	integer	C	0 = The shipping address contact details cannot be modified. 1 = The shipping address contact details can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0 and *canmodifyshippingaddress* is 0.
uselicensekeyforbillingaddress	integer	C	0 = The user account address is used for the billing address. 1 = The address held in the license key is used for the billing address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifybillingaddress	integer	C	0 = The billing address cannot be modified. 1 = The billing address can be modified. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
useemaildestination	integer	C	0 = Send confirmation emails to order billing address. 1 = Send confirmation emails to order shipping address. 2 = Send confirmation emails to order shipping address and bcc order billing address. 3 = Send confirmation emails to order billing address and bcc order shipping address. **NOTE** This option only applies if *defaultaddresscontrol* is 0.
canmodifypassword	integer	M	0 = The password cannot be modified by the user. 1 = The password can be modified by the user.
creditlimit	decimal	O	The credit limit in the currency specified by Taopix as the system currency.
accountbalance	decimal	O	The account balance in the currency specified by Taopix as the system currency.
giftcardbalance	decimal	O	The gift card balance in the currency specified by Taopix as the system currency.
sendmarketinginfo	integer	O	The status of the *Can Send Marketing Information* flag: 0 = No 1 = Yes
addressupdated	integer	M	Must be set to 1

isactive	integer	M	0 = Account not active 1 = Account active

Key:

M	Mandatory
O	Optional
C	Conditional Please read the column description for additional information
A	Address field that may be mandatory. Please refer to the section on address formats for additional information

Updating An Existing Customer Account

Updating an existing customer account first involves identifying the record within the Taopix USERS database. This record id may be provided by the Taopix system or may involve using the Taopix Data API Query API to locate the record. Once the record has been located the Update API can be used to update the data for the relevant record. Although most columns can be modified, the following columns should not be modified otherwise they may cause the system to fail.

webbrandcode
login
customer
groupcode

Taopix Address Formats

The Taopix Portfolio system supports mailing addresses belonging to different countries that may have a different number of fields and validation requirements. When supplying the address information in the user account array the data must be in the correct format for the country specified. The reason for this is that even though the customer account has originated from an external system it will actually be inserted into the Taopix database as a standard customer account so that Taopix can retrieve account data as required.

The address formats for each country are held within the *COUNTRIES*, *COUNTRYREGION* and *COUNTRYREGIONGROUPS* database tables of the Taopix database.

COUNTRIES Database Table

As well as holding the ISO codes and names of countries supported by Taopix it also holds information on the structure of the address.

Within the *COUNTRIES* database table the columns *region*, *displayfields*, *compulsoryfields* and *fieldlabels* hold the important information on the structure of an address.

region

The *region* column specifies if the country is broken down into regions and will either be an empty string for no regions, STATE for the state field being a region and COUNTY for county field being a region. When a country is specified as having regions the standard Taopix address form input changes from an entry field to a dropdown list that contains values from the *COUNTRYREGION* table. Each region has a code and a name that is used to populate the *region*, *regioncode* and the relevant field (*state* / *county*) within the customer account.

displayfields

The *displayfields* column lists the fields that appear for a specific country when editing addresses within Taopix. The data is held as a list of field names separated by the tag *<p>*.

Examples:
1. The address format for the UK is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>add3<p>add4<p>city<p>county
<p>postcode
```

2. The address format for the USA is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>city<p>state<p>county<p>postcode
```

If the *displayfields* column is empty then Taopix defaults to a standard set of fields which is:
```
country<p>firstname<p>lastname<p>company<p>add1<p>add2<p>add3<p>add4<p>city<p>county<p>state
<p>postcode
```

compulsoryfields

The *compulsoryfields* column lists the fields that must be completed when editing addresses within Taopix. The data is held as a list of field names separated by a comma.

Examples:
1. The compulsory fields for the UK is:
```
country,firstname,lastname,add1,city,county,postcode
```

2. The compulsory fields for the USA is:
```
country,firstname,lastname,add1,city,state,postcode
```

If the *compulsoryfields* column is empty Taopix defaults to a standard set of fields which is:
`country,firstname,lastname,add1,city,postcode`

fieldlabels

The fieldlabels column lists the labels used for each display field when editing addresses within Taopix. Although this column is not required in the user account array to contents may help you identify what the fields are used for. The data is held as a list of Taopix language strings separated by a comma.

Examples:
1. The field labels for the UK is:
`str_LabelCountry,str_LabelFirstName,str_LabelLastName, str_LabelLastName, str_LabelLastName,str_LabelCompanyName,str_LabelAddressLine1,str_LabelAddressLine2,str_LabelAddressLine3,str_LabelAddressLine4,str_LabelTownCity,str_LabelCounty,str_LabelPostCode`

2. The field labels for the USA is:
`str_LabelCountry,str_LabelFirstName,str_LabelLastName,str_LabelCompanyName,str_LabelAddressLine1,str_LabelAddressLine2,str_LabelTownCity,str_LabelState,str_LabelCounty,str_LabelZIPCode`

If the *fieldlabels* column is empty then Taopix defaults to a standard set of fields which is:
`str_LabelCountry,str_LabelFirstName,str_LabelLastName,str_LabelCompanyName,str_LabelAddressLine1,str_LabelAddressLine2,str_LabelAddressLine3,str_LabelAddressLine4,str_LabelTownCity,str_LabelCounty,str_LabelState,str_LabelPostCodee`

COUNTRYREGION Database Table

The *COUNTRYREGION* database table contains records for each country that has regions enabled. Within the database table the columns *countrycode*, *regioncode*, *regionname* and *regiongroup* hold the important information for an individual region.

countrycode

The *countrycode* column contains the ISO2 country code that the region entry relates to.

regioncode

The *regioncode* column contains the unique code for the region entry.

regionname

The *regionname* column contains the region name (eg: county name / state name). The data in this column is stored using the Taopix multi-lingual string format but is not stored in the user account in multi-lingual format.

regiongroupcode

When Taopix displays the region dropdown list it is possible to group regions together (ie: regions for England, Scotland, Wales, Northern Ireland). When regions are grouped this column will hold the group code that relates to the group held in the *COUNTRYREGIONGROUPS* database table.

COUNTRYREGIONGROUPS Database Table

The *COUNTRYREGIONGROUPS* database table contains information on how each country's group of regions should be presented. Further knowledge of this data is not required for the purposes of the scripting as it is only used when Taopix displays it's dynamic address form.

CURRENT LIMITATIONS

The following limitations will be addressed based on licensee feedback and requirements:

1. Meta-Data
   It is currently not possible to specify order or component meta-data.